

Infoblox WAPI documentation

Introduction

The Infoblox WAPI is an interface based on REST (REpresentational State Transfer), also called a RESTful web API. It uses HTTP methods for operations and supports input and output in JSON and XML.

Notation

The following conventions are used to describe syntax for WAPI methods and objects:

What	Description
objref	A reference to an object. This must be a reference returned from an earlier call. For more information, see Object Reference .
WAPI	Used as a generic start in an URL. In real calls, this needs to be replaced with <code>/wapi/v2.13.7</code> or similar syntax.
objtype	The name of an object type, such as <code>network</code> .
field	The name of a field, such as <code>comment</code> .
value	The value of an item, such as a field. The value must be quoted according to where it is used. For information, see Naming and Values .
[<i>thing</i>]	These brackets are used to signify an optional value.
<i>a</i> <i>b</i>	The symbol is used to indicate that either <i>a</i> or <i>b</i> can be used.
<i>thing</i> ...	... is used at the end of an item to signify that it can be repeated multiple times. Items must be separated in accordance with where they are used, such as & in arguments.
{ }	These brackets are used to group information in descriptions.

Transport and Authentication

WAPI uses HTTPS ([HTTP](#) over [SSL/TLS](#)) as the transport mechanism. The server certificate used for WAPI is the same certificate used by NIOS for the GUI and PAPI.

WAPI supports both HTTP basic authentication and certificate-based authentication. For certificate-based authentication, see URL [here](#). It is supported to use the connection for multiple requests. In this case, authentication is handled by supplying the cookie (`ibapauth`) that was returned after the initial authentication. This cookie can be invalidated by sending a POST request to `/wapi/v2.13.7/logout`

WAPI supports the same underlying authentication methods that NIOS supports for username and password. All WAPI users must have permissions that grant them access to the API (same as PAPI).

Backward Compatibility

The Infoblox WAPI has a versioning scheme that is independent of the NIOS versioning scheme. The current WAPI version is 2.13.7.

A current WAPI version is backward compatible with WAPI releases that have the same major WAPI version or with designated earlier major versions. Though the protocol itself may not be strictly backward compatible, the server emulates the correct behavior, when necessary.

For example, a client that uses WAPI version *X* behaves the same way in version *Y* if *X* is supported by *Y* (that is *X* is lower than *Y* and *X* has the same major version as *Y* or *X* uses a major version that is supported by *Y*).

The WAPI protocol is versioned (see URL in [General Syntax and Options](#)) independently from NIOS. Refer to the release notes for information about the WAPI version.

Requirements and exceptions:

- Rely on errors returned by HTTP Error Status only, not by text messages or other components.
- New objects and fields may exist in a later WAPI version. Thus, additional fields may be returned and must be ignored.
- New syntaxes and values may be supported. Do not rely on receiving errors for illegal usage.
- In the URL, use the WAPI version that corresponds to the behavior you expect. Do not combine requests using different WAPI versions in the same session or connection.

General Syntax and Options

All WAPI requests consist of three parts; URL, Arguments and Data (body).

URL

The first part of the URL identifies the requests as a WAPI request and specifies the expected version of WAPI. The URL syntax is **wapi/v** *major.minor*, e.g. wapi/v3.4/. The current version of the API is 2.13.7.

The second part of the URL identifies the resource, such as a network, on which the request operates.

Arguments

CGI query arguments (after ?) can be used to specify general options and method specific options and data for the request. All options start with the character **_** (underscore).

The general options are:

Option	Description
_return_type	Data format for returned values; defaults to json . Valid choices: json , json-pretty , xml , xml-pretty . -pretty variants are the same except that they are formatted for readability. For more information, see Data Formats .

Option	Description
<code>_method</code>	An alternative way of specifying HTTP method and overrides the method used. The default is to use the actual HTTP method. Valid choices: GET , PUT , DELETE and POST

Argument *key = value* pairs must be separated with **&**. The values must be quoted using **% xx** notation if they contain the following: **=**, **&**, **+**, **%**, or space.

You can specify only atomic values as arguments (i.e. booleans, integers, or strings). You must use a method that contains a body if lists or structures are needed. Example: POST with `_method=GET` can be used for searching.

In all method descriptions, you can use general options with all requests unless specifically noted.

The methods have additional options as described in their respective sections.

The following table lists the scheduling and approval specific options. Note that you can apply these options only to PUT, POST and DELETE requests.

Option	Description
<code>_schedinfo.scheduled_time</code>	If set, the requested operation will be scheduled for later execution at the specified time (specified in Epoch seconds). A reference to the created scheduledtask object will be returned. Only one of <code>scheduled_time</code> and <code>schedule_now</code> can be set in the request.
<code>_schedinfo.schedule_now</code>	If set to True, the operation will be scheduled for execution at the current time. Note that only <code>scheduled_time</code> or <code>schedule_now</code> can be set in the request.
<code>_schedinfo.predecessor_task</code>	Optional reference to a scheduled task that will be executed before the submitted task.
<code>_schedinfo.warnlevel</code>	Optional warning level for the operation, valid values are 'WARN' and 'NONE'. If not specified, 'NONE' will be used.
<code>_approvalinfo.comment</code>	Comment for the approval operation (this can be optional or required depending on the settings for the approval workflow).
<code>_approvalinfo.query_mode</code>	Optional query mode for the approval operation. Valid values are "true" or "false", if this is set to true and the request would have required approval, an error message will be returned. The default value for this is "false".
<code>_approvalinfo.ticket_number</code>	Ticket number for the approval operation (this can be optional or required depending on the settings for the approval workflow).

Data (Body)

Contains data that is dependent on the method. For information about data format and how to specify it, see [Data Formats](#). Only, PUT, and POST methods can have a Body on input. All methods have Body on output.

Example

The GET request:

```
https://1.2.3.4/wapi/v2.13.7/networkview?
_return_type=xml-pretty&name=default
```

Returns with a body:

```
<?xml version="1.0"?>
<list>
  <value type="object">
    <is_default type="boolean">true</is_default>
    <_ref>networkview/ZG5zLm5ldHdvcmtdmldyQw:default/true</_ref>
    <name>default</name>
  </value>
</list>
```

Naming and Values

WAPI uses a leading underscore (_) for all reserved arguments, fields, and items. Example: **_return_type** and **_ref**.

Fields in objects always start with a letter (a-z) and are followed by a zero or more letters, digits, and underscores. No other characters are used in field identifiers.

Field and argument values must be quoted according to where they are used. Examples:

URL/CGI args, x-www-form-urlencoded:

Use %xx encoding for "%", ",", "/", "?", ":", "@", "&", "=", "+", "\$", " ", and " " (a space)

JSON Data:

Use JSON quoting, as specified at <http://json.org>

XML Data

Use XML quoting (& etc.) as needed for XML.

Values set in WAPI object fields might differ from the effective value used for that particular field during product operation, which could be a value inherited from the Grid or the Grid Member depending on the particular object in question and the state of the object use flags.

Object Reference

WAPI Objects are referenced using their Object References. WAPI returns this reference when an object is created, modified, deleted or read. This reference is used to identify the object for the same operations.

An object reference is a string with the following format, without spaces:

```
wapitype / refdata [ : name1 [ { / nameN }... ] ]
```

Component	Description
wapitype	The object type being referenced. Example: network .
refdata	Opaque internal object identifier. A sequence of letters, digits, “-” (dash) and “_” (underscore).
nameN	Object type dependent name component <i>N</i> . The component describes the object being referenced. This is only returned for objects with a defined name format. It is always optional on input and never used by the server.

The documentation for each object type describes the format of its *name* components. Name components are separated by “/” (or only one component without a “/”). Each name component uses the URL quoting method (%xx notation) when necessary (for example if it contains a “/” character).

If the name is defined for the object type, it can be used by a client to get basic information about an object without retrieving the full object. Example: the name of a host. However, an object’s name is not guaranteed to uniquely identify an object unless specifically noted in its description.

The name is not used by the WAPI server on input, and any supplied value is disregarded. For example, a client is free to send a previously returned reference to the server, with or without the name part, including the leading colon (:). The result is not affected.

Note that non-ascii values in name are returned using % notation, and should be interpreted as hex-encoded utf-8.

Example:

```
record:cname/ZG5 .... DE:t1.webapi16.foo.bar/default
```

Function Calls

Functions are associated with particular objects. The method specific option **_function** should be used to specify the name of function to call. Only POST method allows function calls. You can use either CGI argument key = value pairs or request’s data(body) to specify values for function arguments. Simultaneous use of CGI arguments and data(body) is not supported.

Example 1

The POST request:

```
https://1.2.3.4/wapi/v2.13.7/network/
ZG5zLm5ldHdvcmskMTAuMC4wLjAvMjQvMA:10.0.0.0/24/default?
_function=next_available_ip&num=3
```

Returns with a body:

```
{
  "ips": [
```

```
        "10.0.0.1",  
        "10.0.0.2",  
        "10.0.0.3"  
    ]  
}
```

Example 2

The POST request:

```
https://1.2.3.4/wapi/v2.13.7/network/  
ZG5zLm5ldHdvcmskMTAuMC4wLjAvMjQvMA:10.0.0.0/24/default?  
_function=next_available_ip
```

Sent with a body:

```
{  
  "num": 3  
}
```

Returns with a body:

```
{  
  "ips": [  
    "10.0.0.1",  
    "10.0.0.2",  
    "10.0.0.3"  
  ]  
}
```

Extensible Attributes

Object types that allow for extensible attributes have a field called `extattrs`, which can be read by including the name in the `_return_fields` option of the GET method.

Extensible attributes are sets of name value pairs in which the values can be lists, if the attribute allows for multiple values.

Searching for extensible attributes requires the use of a special syntax, as described under the GET method.

Use Flags

Some fields are associated with a corresponding boolean flag value that has the prefix **use_**. For example, **ttl** is associated with the flag **use_ttl**. In an object, the value of this field will only take effect when its use flag is true. Otherwise, the value will be inherited from a higher level setting.

Use flags and fields that contain the flags behave mostly like other object fields. They are special in the following ways:

- All use flags have names such as “**use_***”, where “*****” is typically the name of the associated field. Multiple fields may share the same use flag.
- Use flags can be read using **_return_fields**.

- If a field is part of the default fields returned on read (“basic object”), its associated use flag (if any) will also be included in the default set.
- Use flags can be written by PUT or POST requests.
- Writing a field that has a corresponding use flag will automatically set the use flag to true, unless the same request also sets the use flag to false.

Data Formats

Input

The body of the HTTP request contains data for the PUT and POST requests only. The format of the data defaults to JSON, but it can be changed using Content-Type: header. The valid content types are:

Content Type	Description
application/json	JSON format, see http://json.org for more information.
application/xml	XML format, see XML Format for more information.
text/xml	Alternative way to specify application/xml.
application/x-www-form-urlencoded	Arguments to method encoded in body. This is the same as specification after <code>?</code> , but it can handle longer sequences and is directly supported by HTML forms. If arguments are encoded in the body, CGI query arguments won't be allowed.

Output

Data returned to the client defaults to JSON, but can be changed using either Accept: header or **_return_type**. Accept: takes the same values as Content-Type, listed above (for exceptions to this, see Error Handling); **_return_type** overrides any Accept: header.

XML Format

WAPI uses the following XML constructs:

Element	Description
<list>	Array, child nodes are items in list. Names of child elements are not significant (and can be same).
<X type="object">	Object X, child nodes are members of object. X can be any value if used outside an object context
<X type="T">	Field X of object. Its value is the text of the element. Allowed types (T) are int, float, boolean and string (as in XML Schema Definition). String is the default and is not explicitly specified using <i>type=</i> on output.
<X null="true"/>	Field X with value null/None.

Field syntax is used for “bare” values in list/array or as single values. *X* is not significant and will always be **value** on output.

No name spaces are used or specified.

Example: XML (xml-pretty style):

```
<?xml version="1.0"?>
<list>
  <value type="object">
    <comment null="true"/>
    <_ref>network/ZG5zLm5ldHdvcmSkMTAuMC4wLjAvOC8w:20.0.0.0/8/default</_ref>
    <disable type="boolean">false</disable>
    <network>20.0.0.0/8</network>
    <network_view>default</network_view>
  </value>
</list>
```

If *X* is considered an illegal XML tag name, or if it begins with “tag” and is followed by a number it will be renamed to tag0-N and an additional “name” property will be added on retrieval and expected on input. For example, the XML for an object with extensible attributes that contain spaces in their names would look like the following:

```
<?xml version="1.0"?>
<list>
  <value type="object">
    <network>8.0.0.0/8</network>
    <extattrs type="object">
      <tag0 name="12345" type="object">
        <value>d</value>
      </tag0>
      <tag1 name="tag0" type="object">
        <value>c</value>
      </tag1>
      <tag2 name="this is a test" type="object">
        <value>b</value>
      </tag2>
    </extattrs>
    <_ref>network/ZG5zLm5ldHdvcmSkOC4wLjAuMC84LzA:8.0.0.0/8/default</_ref>
  </value>
</list>
```

Error Handling

All errors return a HTTP status code of 400 or higher.

All methods use the following generic error status codes. Specific return codes used for a method are specified for each method.

Status	Description
--------	-------------

400	Bad Request. The request from the client is incorrect. This could be syntax errors in the request or data conflict issues on the server side. The request should not be repeated as is unless the error condition has been cleared (i.e. either the request syntax corrected or the state of the database changed.)
500	Server Error. The error was not caused by any error in the request. Depending on the error the request may be successfully repeated as is. If not possible to resolve, please report to Infoblox (including the full error return with the “trace”).

4xx codes refer to errors caused by the request or the data. To some extent, all of these are user errors.

5xx codes refer to server or internal errors. These errors point to deficiency in the server code and are not usually possible under normal conditions.

When the server returns an error with status code ≥ 400 , the body is always in JSON format, irrespective of any `Accept` or `_return_types`.

The returned message conforms to JSON, but is formatted to ensure that the first line of the body always contains the text “Error,” an error type, and an error message.

A client that only gives a description of the error can simply show the first returned line.

The full returned error data is an object with the following fields (all values are strings):

Field	Value
Error	Error type (followed by an explanation after :).
code	Symbolic error code.
text	Explanation of the error.
trace	Debug trace from the server, only if debug is on.

Example of Error Return (trace shortened):

```
{ "Error": "AdmConProtoError: Unknown argument/field: netwdork",
  "code": "Client.Ibap.Proto",
  "text": "Unknown argument/field: netwdork",
  "trace": "  File \"/infoblox/common/lib/python/info..."
}
```

Methods

GET

Search and Read Objects: GET Method

HTTP GET is used to read a single object or to search for objects.

Syntax

```
GET WAPI / objref [ ? option... ]
```

or

```
GET WAPI / objtype [ ? { option | condition }... ]
```

Description

GET is used to read objects. The objects to read can be specified either by using an [Object Reference](#) (*objref*) to read one specific object or by searching for objects of a specific type (*objtype*) with the given search conditions.

Arguments to the search (*objtype*) form are field names and values to match. If no arguments are used, all object for the object type *objtype* are returned.

The number of objects returned is limited by the option `_max_results` or, if `_max_results` is not specified, 1000 objects. If `_max_results` is not specified, the appliance returns an error when the number of returned objects would exceed 1000. Similarly, if `_max_results` is set to -500 (maximum of 500 objects) the appliance returns an error if the number of returned objects would exceed 500.

Options

Method Option	Description
<code>_max_results</code>	Maximum number of objects to be returned. If set to a negative number the appliance will return an error when the number of returned objects would exceed the setting. The default is -1000. If this is set to a positive number, the results will be truncated when necessary.
<code>_return_fields</code>	List of returned fields separated by commas. The use of <code>_return_fields</code> repeatedly is the same as listing several fields with commas. The default is the basic fields of the object.
<code>_return_fields+</code>	Specified list of fields (comma separated) will be returned in addition to the basic fields of the object (documented for each object).
<code>_return_as_object</code>	If set to 1, a results object will be returned (see below for more information). If not specified, it defaults to 0.
<code>_paging</code>	If set to 1, the request is considered a paging request (see below for more information). If not specified, it defaults to 0. If set, <code>_max_results</code> must also be set.
<code>_page_id</code>	If set, the specified page of results will be returned.
<code>_proxy_search</code>	If set to 'GM', the request is redirected to Grid master for processing. If set to 'LOCAL', the request is processed locally. This option is applicable only on vConnector grid members. The default is 'LOCAL'.
<code>_schema</code>	If this option is specified, a WAPI schema will be returned (see below for more information).
<code>_schema_version</code>	If this option is specified, a WAPI schema of particular version will be returned. If options is omitted, schema version is assumed to be 1. For the full list of available versions please refer to information below.

Method Option	Description
<code>_get_doc</code>	If this option is specified, a WAPI schema with documentation will be returned. Applicable only when <code>_schema_version</code> is 2.
<code>_schema_searchable</code>	If this option is specified, search only fields will also be returned. Applicable only when <code>_schema_version</code> is 2.
<code>_inheritance</code>	If this option is set to True, fields which support inheritance, will display data properly.

Arguments

There can be no arguments to `objtype` or it can have one or multiple conditions in the following format:

```
{ field | * attribute [ <space> ] } [ modifiers ] = value
```

Where:

`field` is a documented field of the object.

`attribute` is the name of an extensible attribute. Must be prefixed by an asterisk (*) and optionally followed by a single space.

`modifiers` is optional and can be one or more search modifiers supported by the field or extensible attribute value type.

`value` is the value or regular expression to search for.

When combining multiple conditions, all must be satisfied in order to match an object (i.e. conditions are combined with AND).

When a field is a list or an extensible attribute that can have multiple values, the condition is true if any value in the list matches.

If no modifiers are used, it is an exact match.

Search Modifiers

A search argument can use the following modifiers:

Modifier	Functionality
!	Negates the condition.
:	Makes string matching case insensitive.
~	Regular expression search. Expressions are unanchored.
<	Less than or equal.
>	Greater than or equal.

Only one of the following can be specified at one time: greater than, less than, and regular expressions.

You can find the modifiers that are supported by each field in the respective documentation. Unsupported combinations will result in an error.

Depending on the attribute type, following are modifiers supported by extensible attributes:

integer and date support `!`, `<` and `>`. All other types behave like strings and support `!`, `~` and `:`.

Data Returned

In the object reference form (*objref*) only one object is returned (as an object, not a list). In the search form (*objtype*) the request always returns a list of objects (even if zero or one objects is returned).

Objects returned will by default consist of a set of basic fields, as listed in the documentation. The option **`_return_fields`** can be used to request a specific set of fields to return.

Fields that have no value (not set in the NIOS database) or that are not allowed to be accessed by the user because of group access rights will not be returned (i.e. silently left out of the result).

Returned objects will also contain a **`_ref`** field, containing the reference of the object. This can be used in subsequent calls that require a reference.

If a search matches no objects, an empty list will be returned.

If a results object is requested, an object with the following fields will be returned:

Field	Present	Description
result	Always	Actual result of the read operation, this is a list of objects.
next_page_id	Optional	If there was a paging request, this is the ID for the next page of results.

Some fields refer to other subobjects. Some of these fields also support nested return fields (see the field's 'Type' section for more information). In the case of nested return fields, you can request specific fields of the subobject by concatenating them to the parent field using the `'.'` (period) character.

For example, during a search for `record:host`, you can request the return of the `'bootserver'` field in subobject `'ipv4addrs'` by passing a return field in the form of `'ipv4addrs.bootserver'`. You can also specify subobject fields as part of a `_return_fields+` invocation. In this case, the specified return field will be returned in addition to the standard fields for the specified subobject.

If an empty subobject field is passed, and the subobject field is a reference-only nest return field, it is equivalent to asking for the standard

fields of that subobject. This can be useful if the subobject field returns only the reference of the subobject by default. For example, in the 'permission' object, the 'object' field normally contains only the reference of the object to which the permission applies. To request the standard fields for the object, you must explicitly reference the field name preceded by the keyword "object" and a period "." For example, `_return_fields=object.fqdn`

If a field can support multiple object types, for example 'record' inside allrecords, only fields common to all the multiple object types should be specified as subobject fields. Otherwise if a subobject for which the subfield is not valid exists, an error would be returned.

Return Status/Errors

Status	Description
200	Normal return. Referenced object or result of search in body.
400	Results set would contain more than <code>_max_results</code> objects (only generated if <code>_max_results</code> is negative).
404	Referenced object not found (if <i>objref</i> form is used, empty list and 200 is returned for empty search result)

Results paging

For searches that return a large number of results, paging is desirable.

To start a paging request, the initial search request must have `_paging` and `_return_as_object` set to 1, and `_max_results` set to the desired page size.

The server will then return a results object that contains the `next_page_id` field and the result field set to the first page of results.

Note that the `next_page_id` field only contains URL-safe characters so it can be used as is and no quotation characters are required for subsequent requests.

To get more results, you should send GET requests to the original object and set `_page_id` to the ID string returned in the previous page of results.

The server does not return a `next_page_id` field in the last page of results. Paging requests are considered independent requests, so the set of results might change between requests if objects are added or removed from the server at the same time when the requests are occurring.

For an invocation example, see the sample code section in the manual [here](#).

WAPI Schema Fetching

If the `_schema` option is passed, the request will execute a schema fetch. Other options, such as `_max_results`, `_return_fields`, etc., will be ignored.

The WAPI schema returned in the format requested using either the Accept: header or `_return_type` as specified by WAPI.

Note that this is not intended to be a schema as defined by JSON or XML standards.

If a WAPI schema is requested using the **_schema** option without specifying **objtype**, an object with the following fields will be returned:

Field	Description
requested_version	Actual requested version of the WAPI schema.
supported_objects	List of supported objects in the requested version.
supported_versions	List of all supported versions.

Example. Use a GET request to get the WAPI schema:

```
https://1.2.3.4/wapi/v1.0/?_schema
```

Returns with a body (lists shortened):

```
{ "requested_version": "1.0",
  "supported_objects": ["ipv4address", "ipv6address", "ipv6network",
                        "ipv6networkcontainer", "ipv6range",
                        "macfilteraddress", "network", ...],
  "supported_versions": ["1.0", "1.1", "1.2", "1.2.1", ...]
}
```

If the described above is done specifying **_schema_version=2**, then following field will be returned additionally:

Field	Description
schema_version	The version of schema description requested.
supported_schema_versions	List of supported versions for schema description.

Example:

```
https://1.2.3.4/wapi/v2.5/?_schema=1&_schema_version=2
```

Returns with a body (lists shortened):

```
{ "requested_version": "2.5",
  "schema_version": "2",
  "supported_schema_versions": ["1", "2", ],
  "supported_objects": ["ad_auth_service", ... ],
  "supported_versions": ["2.3", "2.5", ... ]
}
```

If the *objtype* is specified for WAPI schema fetching, an object with the following fields will be returned:

Field	Description
cloud_additional_restrictions	List of cloud restrictions.
fields	List of fields of the object.
restrictions	List of object restrictions.

Field	Description
type	Requested <i>objtype</i> .
version	Actual requested version of the WAPI object schema.

The fields specific to schema description #2:

Field	Description
schema_version	The version of schema description requested.
wapi_primitive	Determines if the requested WAPI primitive is object, structure or function call.

The list of object restrictions that contain supported operations for the object. Example of operations: “create”, “delete”, “read”, “update”, “function call”, “permissions”, “global search”, “scheduling”, “csv”.

The `cloud_additional_restrictions` field contains the list of additional unsupported operations when using Cloud Network Automation. Example of operations: “all”, “create”, “delete”, “read”, “update”, “function call”, “permissions”, “global search”, “scheduling”, “csv”.

The returned **fields** list is composed by individual objects each describing a field of the API object. These objects have the following members:

Parameter	Description
is_array	True if this field is an array.
name	Name of this field.
searchable_by	String with supported search modifiers: “=”, “!”, “:”, “~”, “<”, “>”.
standard_field	True for fields that are returned by default.
supports	List of supported operations: “s”, “w”, “u”, “r”.
type	List of supported types.
wapi_primitive	Determines if the requested WAPI primitive is object, structure or function call.

The fields specific to schema description #2:

Field	Description
schema_version	The version of schema description requested.
wapi_primitive	Determines if the requested WAPI primitive is object, structure or function call.
supports_inline_funcall	Determines if the field can be initialized by calling an inline function.

Field	Description
doc	The documentation of this field. It's applicable only when _get_doc=1 is used. The returned documentation string might contain ReStructuredText directives.

The version #2 delivers all information regarding structures and function calls.

Please keep in mind that **enum_values** is changed in #2. It cannot be a dictionary, as it was in #2, but a list.

Example. Use a GET request to get the 'networkview' WAPI object schema for WAPI version 1.4:

```
https://1.2.3.4/wapi/v1.4/networkview?_schema
```

Returns with a body (lists shortened):

```
{ "cloud_additional_restrictions": ["all"],
  "fields": [{
    "is_array": false,
    "name": "comment",
    "searchable_by": ":",
    "standard_field": true,
    "supports": "rwus",
    "type": ["string"]
  }, {
    "is_array": false,
    "name": "name",
    "searchable_by": ":",
    "standard_field": true,
    "supports": "rwus",
    "type": ["string"]
  }, ...],
  "restrictions": ["scheduling", "csv"],
  "type": "networkview",
  "version": "1.4"
}
```

Example of new information for version #2 (the same request as above but different **objtype** and HTTP arguments:

```
https://1.2.3.4/wapi/v2.13.7/grid?_schema=1&
_schema_version=2&_get_doc=1
```

Returns with a body (lists shortened and cut):

```
{ "doc": "Test connectivity to the REST API endpoint.",
  "is_array": false,
  "name": "test_connection",
  "schema": {
    "input_fields": [],
    "output_fields": [
      { "doc": "The overall status of connectivity test.",
        "enum_values": [
          "FAILED",
```

```

        "SUCCESS"
    ],
    "is_array": false,
    "name": "overall_status",
    "supports": "r",
    "type": ["enum"]
},
{ "doc": "The test connectivity failed error message.",
  "is_array": false,
  "name": "error_message",
  "supports": "r",
  "type": ["string"]
}
],
},
"standard_field": false,
"supports": "rwu",
"type": ["testconnectivityparams"],
"wapi_primitive": "funcall"
},
{ "doc": "The notification REST template instance. The parameters
    REST API endpoint template instance are prohibited to
    change.",
  "is_array": false,
  "name": "template_instance",
  "schema": {
    "fields": [
      { "doc": "The name of REST API template parameter.",
        "is_array": false,
        "name": "template",
        "supports": "rwu",
        "type": ["string"]
      },
      { "doc": "The notification REST template parameters.",
        "is_array": true,
        "name": "parameters",
        "supports": "rwu",
        "type": ["notification:rest:templateparameter"]
      }
    ]
  },
  "standard_field": false,
  "supports": "rwu",
  "type": ["notification:rest:templateinstance"],
  "wapi_primitive": "struct"
},

```

WAPI Inheritance Data Fetching

If the **_inheritance** option is passed and set to True, the request will fetch inheritance data. Inheritance data will only show for fields which were queried and support this mode. Inheritance support started from version 2.10.2.

In case of basic inheritance, an object with following fields will be returned:

Field	Description
inherited	Flag to display whether value was inherited or not.
multisource	Flag to display if value was inherited from multiple sources.

Field	Description
source	String containing WAPI reference to source of data. Empty string if data belongs to a queried object.
value	Actual value.

Example. Use a GET request to get the 'grid' object's 'ipam_threshold_settings' and inheritance info, if needed:

```
https://1.2.3.4/wapi/v2.10.3/network?_inheritance=True
&_return_fields=ipam_threshold_settings
```

Output when object's data is shown:

```
{
  "ipam_threshold_settings": {
    "inherited": false,
    "multisource": false,
    "source": "",
    "value": {
      "reset_value": 86,
      "trigger_value": 96
    }
  }
},
```

Output when inherited data is shown:

```
{
  "ipam_threshold_settings": {
    "inherited": true,
    "multisource": false,
    "source": "grid/b25lLmNsdXN0ZXIkMA:Infoblox",
    "value": {
      "reset_value": 85,
      "trigger_value": 95
    }
  }
},
```

In case of multiple inheritance, array of values with their sources will be shown. An object with following fields will be returned:

Field	Description
inherited	Flag to display whether value was inherited or not.
multisource	Flag to display if value was inherited from multiple sources.
values	List of structs, describing values, inherited from multiple source. Structure described below.

Structure of 'values' object:

Field	Description
source	String containing WAPI reference to source of data. Cannot be empty.
value	Actual value.

Example of query:

```
https://1.2.3.4/wapi/v2.10.3/network?_inheritance=True
    &_return_fields=pxe_lease_time
```

Output:

```
{
  "pxe_lease_time": {
    "inherited": true,
    "multisource": true,
    "values": [
      {
        "source": "grid:dhcpproperties/ZG5zLmNX9wZXJ0aWVzJDA:Infoblox",
        "value": 43200
      },
      {
        "source": "member:dhcpproperties/ZG5zMkMA:infoblox.localdc",
        "value": 403200
      }
    ]
  }
},
```

In case if the object of inheritance is a list consisting of structs and every item of this list is inherited independently, items will be grouped by their sources. Some items may appear several times if they are inherited from multiple sources.

Structure of such groups is described below:

Field	Description
inherited	Flag to display whether value was inherited or not.
source	String containing WAPI reference to source of data. Empty string if data belongs to a queried object.
value	List of structs, inherited from given source.

Example of query:

```
https://1.2.3.4/wapi/v2.10.3/network?_inheritance=True
    &_return_fields=options
```

Output:

```
{
  "options": [
    {
      "inherited": true,
      "source": "member:dhcpproperties/ZG5zL1lByb3BlcnRpZXMKMQ:member",
      "values": [
        {
          "name": "subnet-mask",
          "num": 1,
          "value": "255.255.254.0",
          "vendor_class": "DHCP"
        }
      ]
    }
  ],
},
```

```
{
  "inherited": true,
  "source": "member:dhcpproperties/ZG5zLXMkMA:infoblox.localdc",
  "values": [
    {
      "name": "subnet-mask",
      "num": 1,
      "value": "255.255.255.0",
      "vendor_class": "DHCP"
    }
  ]
},
{
  "inherited": true,
  "source": "grid:dhcpproperties/ZG5zLmNXN0Z9wZXJ0aWVzJDA:Infoc",
  "values": [
    {
      "name": "dhcp-lease-time",
      "num": 51,
      "use_option": false,
      "value": "43200",
      "vendor_class": "DHCP"
    }
  ]
}
],
},
```

POST

Create Object: POST Method

The POST method is used to create a new object. It can also be used for all other operations via the [the wapi object](#)

Syntax

POST *WAPI / objtype* [? { *options* | *field = value* }...]

Description

The data for the request must contain all required fields of the *objtype*. Data can be given as arguments as shown above or as the body of the request (but not both).

Options

Method Option	Description
<code>_return_fields</code>	A list of returned fields separated by commas. The use of <code>_return_fields</code> repeatedly is the same as listing several fields with commas. The default is the basic fields of the object.
<code>_return_fields+</code>	Specified list of fields (comma separated) will be returned in addition to the basic fields of the object (documented for each object).
<code>_inheritance</code>	If this option is set to True, fields which support inheritance, will display data properly.

Options can be given only as query arguments as shown above, they cannot be included in the body of the request.

Arguments

Arguments can be used to supply the object instead of using the body.

Data (Body)

Data for object to be created. Can be used as alternative to arguments. All fields marked as required for the object must be supplied. All fields not supplied will be defaulted as specified for the object. See [Use Flags](#) for information about special handling for these fields.

Data Returned

[Object Reference](#) of the object created, returned as a string.

If required, specify the ‘_return_fields’ option to examine the values of fields that were set by the appliance as part of the insertion. It is possible for the appliance to return the newly inserted object, instead of a reference string.

Passing an empty value to the ‘_return_fields’ option will cause only the object reference to be set inside the returned object. Passing an empty value to the ‘_return_fields+’ option will cause the returned object to contain its standard fields. Passing any other values will return the specified fields.

Return Status/Errors

Status	Description
201	Object created (success)

PUT

Update Object: PUT Method

The PUT method is used to update an existing object. The syntax of PUT is:

Syntax

PUT *WAPI / objref* [? { *option* | *field = value* }...]

Description

PUT is used to update an existing object (given by the [Object Reference](#), *objref* in the request). Only the fields supplied are updated (except as described for [Use Flags](#)).

Options

Method Option	Description
_return_fields	List of returned fields separated by commas. The use of _return_fields repeatedly is the same as listing several fields with commas. The default is the basic fields of the object.
_return_fields+	Specified list of fields (comma separated) will be returned in addition to the basic fields of the object (documented for

Method Option	Description
	each object).
<code>_inheritance</code>	If this option is set to True, fields which support inheritance, will display data properly.

Options can be given only as query arguments as shown above, they cannot be included in the body of the request.

Arguments

The data to be updated can be given as argument as shown in the syntax or as the body of the request (but not both).

Data (Body)

Data for object to be updated. Can be used as alternative to arguments.

Data Returned

[Object Reference](#) of the object modified, returned as a string.

The object reference may have been changed by the operation.

If required, specify the ‘_return_fields’ option to examine the values of fields that were set by the appliance as part of the update. It is possible for the appliance to return the newly updated object, instead of a reference string.

Passing an empty value to the ‘_return_fields’ option will cause only the object reference to be set inside the returned object. Passing an empty value to the ‘_return_fields+’ option will cause the returned object to contain its standard fields. Passing any other values will return the specified fields.

Return Status/Errors

Status	Description
200	Object updated (success)

DELETE

Delete Object: DELETE Method

The DELETE method is used to delete an object.

Syntax

DELETE *WAPI / objref [? option...]*

Description

DELETE is used to delete an existing object (given by the [Object Reference](#), *objref* in the request).

Options

There are no DELETE specific options.

Arguments

There are no general DELETE arguments. Some of the objects has object-specific DELETE arguments, which are described in the ‘Delete arguments’ section of their respective documentation.

Data Returned

Returns the [Object Reference](#) of the deleted object as a string.

Return Status/Errors

Status	Description
200	Object deleted (success)

Objects

- [ad_auth_service](#) : Active Directory Authentication Service object.
- [admingroup](#) : Admin Group object.
- [adminrole](#) : Admin Role object.
- [adminuser](#) : Admin User object.
- [allendpoints](#) : All Endpoints object.
- [allnsgroup](#) : All NS Group object.
- [allrecords](#) : AllRecords object.
- [allrpzrecords](#) : DNS All RPZ Records object.
- [approvalworkflow](#) : The approval workflow object.
- [authpolicy](#) : The authentication policy object.
- [awsrte53taskgroup](#) : AWS Route53 task group object.
- [awsuser](#) : AWS User object.
- [azurednstaskgroup](#) : Azure DNS task group object.
- [azureuser](#) : Azure User object.
- [bfdtemplate](#) : BFD template object.
- [bulkhost](#) : Bulkhost object.
- [bulkhostnametemplate](#) : The bulk host name template object.
- [cacertificate](#) : CA Certificate object.
- [capacityreport](#) : Grid member capacity report object.
- [captiveportal](#) : Captive portal object.
- [certificate:authservice](#) : Certificate authentication service object.
- [csvimporttask](#) : CSV Import task object.
- [datacollectioncluster](#) : DATA collection cluster object.
- [db_objects](#) : The DB Objects object.
- [dbsnapshot](#) : The DBSnapshot WAPI object.
- [ddns:principalcluster](#) : DDNS Principal Cluster object.
- [ddns:principalcluster:group](#) : DDNS Principal Cluster Group object.
- [deleted_objects](#) : The Deleted Objects object.
- [dhcp:statistics](#) : DHCP Statistics object.
- [dhcpfailover](#) : DHCP Failover Association object.
- [dhcptiondefinition](#) : DHCP option definition object.
- [dhcpoptionspace](#) : DHCP option space object.
- [discovery](#) : Discovery object.
- [discovery:credentialgroup](#) : The Credential group object.
- [discovery:device](#) : Discovery Device object.
- [discovery:devicecomponent](#) : Device Component object.

- [discovery:deviceinterface](#) : Device Interface object.
- [discovery:deviceneighbor](#) : Device Neighbor object.
- [discovery:devicesupportbundle](#) : Device support bundle object.
- [discovery:diagnostictask](#) : The discovery diagnostic task object.
- [discovery:gridproperties](#) : The Grid discovery properties object.
- [discovery:memberproperties](#) : The Grid discovery member properties object.
- [discovery:sdnnetwork](#) : The SDN network object.
- [discovery:status](#) : Discovery Status object.
- [discovery:vrf](#) : The VRF object.
- [discoverytask](#) : The discovery task object.
- [distributionschedule](#) : Distribution schedule object.
- [dns64group](#) : DNS64 synthesis group object.
- [dtc](#) : DTC object.
- [dtc:allrecords](#) : DTC AllRecords object.
- [dtc:certificate](#) : DTC Certificate object.
- [dtc:lbdn](#) : DTC LBDN object.
- [dtc:monitor](#) : DTC monitor object.
- [dtc:monitor:http](#) : DTC HTTP monitor object.
- [dtc:monitor:icmp](#) : DTC ICMP monitor object.
- [dtc:monitor:pdp](#) : DTC PDP monitor object.
- [dtc:monitor:sip](#) : DTC SIP monitor object.
- [dtc:monitor:snmp](#) : DTC SNMP monitor object.
- [dtc:monitor:tcp](#) : DTC TCP monitor object.
- [dtc:object](#) : DTC object.
- [dtc:pool](#) : DTC Pool object.
- [dtc:record:a](#) : DTC A Record object.
- [dtc:record:aaaa](#) : DTC AAAA Record object.
- [dtc:record:cname](#) : DTC CNAME Record object.
- [dtc:record:naptr](#) : DTC NAPTR Record object.
- [dtc:record:srv](#) : DTC SRV Record object.
- [dtc:server](#) : DTC Server object.
- [dtc:topology](#) : DTC Topology object.
- [dtc:topology:label](#) : DTC Topology Label object.
- [dtc:topology:rule](#) : DTC Topology Rule object.
- [dxl:endpoint](#) : The Data Exchange Layer endpoint object.
- [extensibleattributedef](#) : Extensible Attribute Definition object.
- [federatedrealms](#) : Federated realms object.
- [fedipamop](#) : Federated IPAM operations object.
- [fileop](#) : File operations object.
- [filterfingerprint](#) : DHCP Fingerprint Filter object.
- [filtermac](#) : DHCP MAC Address Filter object.
- [filternac](#) : DHCP NAC Filter object.
- [filteroption](#) : DHCP filter option object.
- [filterrelayagent](#) : The filter relay agent object.
- [fingerprint](#) : DHCP Fingerprint object.
- [fixedaddress](#) : DHCP Fixed Address object.
- [fixedaddresstemplate](#) : The fixed address template object.
- [ftpuser](#) : FTP user object.
- [gcpdnstaskgroup](#) : Gcp DNS task group object.
- [gcpuser](#) : GCP User object.

- `gmcgroup` : GMC group object.
- `gmcschedule` : GMC schedule object.
- `grid` : Grid object.
- `grid:cloudapi` : Grid Cloud API object.
- `grid:cloudapi:cloudstatistics` : Grid Cloud Statistics object.
- `grid:cloudapi:tenant` : Grid Cloud API Tenant object.
- `grid:cloudapi:vm` : Grid Cloud API vm object.
- `grid:cloudapi:vmaddress` : Grid Cloud API VM address object.
- `grid:dashboard` : Grid Dashboard object.
- `grid:dhcpproperties` : Grid DHCP properties object.
- `grid:dns` : Grid DNS properties object.
- `grid:filedistribution` : Grid file distribution object.
- `grid:license_pool` : Grid License Pool object.
- `grid:license_pool_container` : Grid License Pool Container object.
- `grid:maxminddbinfo` : Topology DB Info object.
- `grid:member:cloudapi` : Member Cloud API object.
- `grid:servicerestart:group` : Service Restart Group object.
- `grid:servicerestart:group:order` : Restart Group Order object.
- `grid:servicerestart:request` : Restart Request object.
- `grid:servicerestart:request:changedobject` : Grid service restart request changed object.
- `grid:servicerestart:status` : Restart Status object.
- `grid:threatinsight` : Grid threat insight object.
- `grid:threatprotection` : The Grid threat protection object.
- `grid:x509certificate` : X509Certificate object.
- `hostnamerewritepolicy` : Hostname rewrite policy object.
- `hsm:allgroups` : All Hardware Security Module groups object.
- `hsm:entrustnshieldgroup` : The Entrust nShield Harware Security Module group object.
- `hsm:safenetgroup` : The Hardware Security Module SafeNet group object.
- `hsm:thalesgroup` : The Thales Harware Security Module group object.
- `hsm:thaleslunagroup` : The Hardware Security Module Thales Luna group object.
- `ipam:statistics` : IPAM statistics object.
- `ipv4address` : IPAM IPv4Address object.
- `ipv6address` : IPAM IPv6Address object.
- `ipv6dhcptiondefinition` : DHCP IPv6 option definition object.
- `ipv6dhcptionspace` : DHCP IPv6 option space object.
- `ipv6filteroption` : DHCP filter option object.
- `ipv6fixedaddress` : DHCP IPv6 Fixed Address object.
- `ipv6fixedaddresstemplate` : The IPv6 fixed address template object.
- `ipv6network` : DHCP IPv6Network object.
- `ipv6networkcontainer` : DHCP IPv6NetworkContainer object.
- `ipv6networktemplate` : DHCP IPv6 network template object.
- `ipv6range` : DHCP IPv6 Range object.
- `ipv6rangetemplate` : IPv6 DHCP Range template object.
- `ipv6sharednetwork` : DHCP IPv6 Shared Network object.
- `kerberoskey` : Kerberos key object.
- `ldap_auth_service` : The LDAP authentication service object.
- `lease` : DHCP Lease object.
- `license:gridwide` : Gridwide license object.
- `localuser:authservice` : Local user authentication service object.
- `macfilteraddress` : MAC Filter Address object.

- `mastergrid` : Master Grid object.
- `member` : Member object.
- `member:dhcpproperties` : Member DHCP properties object.
- `member:dns` : Member DNS object.
- `member:filedistribution` : Grid member file distribution object.
- `member:license` : Member License object.
- `member:parentalcontrol` : Member mobile security properties object.
- `member:threatinsight` : Grid member threat insight object.
- `member:threatprotection` : Member threat protection object.
- `membercloudsync` : MemberCloudSync object.
- `memberdfp` : Memberdfp object.
- `msserver` : Microsoft Server object.
- `msserver:adsites:domain` : Active Directory Domain object.
- `msserver:adsites:site` : Active Directory Site object.
- `msserver:dhcp` : Microsoft Server DHCP properties object.
- `msserver:dns` : Microsoft Server DNS properties object.
- `mssuperscope` : Microsoft DHCP superscope object.
- `multiregions` : MultiRegions object.
- `namedacl` : Named ACL object.
- `natgroup` : Network Address Translation group object.
- `network` : DHCP Network object.
- `network_discovery` : Network discovery object.
- `networkcontainer` : DHCP Network Container object.
- `networktemplate` : DHCP Network template object.
- `networkuser` : Network User object.
- `networkview` : DHCP NetworkView object.
- `notification:rest:endpoint` : The notification RESTful endpoint object.
- `notification:rest:template` : The notification REST template object.
- `notification:rule` : Notification rule object.
- `nsgroup` : DNS name server group object.
- `nsgroup:delegation` : NS group delegation object.
- `nsgroup:forwardingmember` : Forwarding Member Name Server Group object.
- `nsgroup:forwardstubserver` : Forward Stub Server Name Server Group object.
- `nsgroup:stubmember` : Stub Member Name Server Group object.
- `orderedranges` : Ordered DHCP ranges object.
- `orderedresponsepolicyzones` : Ordered Response Policy Zones object.
- `outbound:cloudclient` : OutBoundCloudClient object.
- `parentalcontrol:avp` : The parental control AVP object.
- `parentalcontrol:blockingpolicy` : Parental control blocking policy object.
- `parentalcontrol:subscriber` : The parental control subscriber object.
- `parentalcontrol:subscriberrecord` : Parental control subscriber record object.
- `parentalcontrol:subscribersite` : Subscriber site parental control properties object.
- `permission` : Permissions object.
- `pxgrid:endpoint` : The PXGrid endpoint object.
- `radius:authservice` : The RADIUS authentication service object.
- `range` : DHCP Range object.
- `rangetemplate` : DHCP Range template object.
- `record:a` : DNS A record object.
- `record:aaaa` : DNS AAAA record object.
- `record:alias` : DNS Alias record object.

- `record:caa` : DNS CAA record object.
- `record:cname` : DNS CNAME record object.
- `record:dhcid` : DNS DHCID record object.
- `record:dname` : DNS DNAME record object.
- `record:dnskey` : DNS DNSKEY record object.
- `record:ds` : DNS DS record object.
- `record:dtclbdn` : DTC LBDN object.
- `record:host` : DNS Host record object.
- `record:host_ipv4addr` : IPv4 Host address object.
- `record:host_ipv6addr` : IPv6 Host address object.
- `record:https` : DNS HTTPS record object.
- `record:mx` : DNS MX record object.
- `record:naptr` : DNS NAPTR record object.
- `record:ns` : DNS NS record object.
- `record:nsec` : DNS NSEC record object.
- `record:nsec3` : DNS NSEC3 record object.
- `record:nsec3param` : DNS NSEC3 record object.
- `record:ptr` : DNS PTR record object.
- `record:rpz:a` : Response Policy Zone Substitute A Record Rule object.
- `record:rpz:a:ipaddress` : Response Policy Zone Substitute IPv4 Address Rule object.
- `record:rpz:aaaa` : Response Policy Zone Substitute AAAA Record Rule object.
- `record:rpz:aaaa:ipaddress` : Response Policy Zone Substitute IPv6 Address Rule object.
- `record:rpz:cname` : DNS Response Policy Zone CNAME record object.
- `record:rpz:cname:clientipaddress` : DNS RPZ CNAMEClientIpAddress record object.
- `record:rpz:cname:clientipaddresssdn` : Substitute Domain Name Based on Client IP Address rule object.
- `record:rpz:cname:ipaddress` : DNS RPZ CNAMEIpAddress record object.
- `record:rpz:cname:ipaddresssdn` : Substitute Domain Name Based on IP Address rule object.
- `record:rpz:https` : Response Policy Zone Substitute HTTPS Record Rule object.
- `record:rpz:mx` : Response Policy Zone Substitute MX Record Rule object.
- `record:rpz:naptr` : Response Policy Zone Substitute NAPTR Record Rule object.
- `record:rpz:ptr` : Response Policy Zone Substitute PTR Record Rule object.
- `record:rpz:srv` : Response Policy Zone Substitute SRV Record Rule object.
- `record:rpz:svcb` : Response Policy Zone Substitute SVCB Record Rule object.
- `record:rpz:txt` : Response Policy Zone Substitute TXT Record Rule object.
- `record:rrsig` : DNS RRSIG record object.
- `record:srv` : DNS SRV record object.
- `record:svcb` : DNS SVCB record object.
- `record:tlsa` : DNS TLSA record object.
- `record:txt` : DNS TXT record object.
- `record:unknown` : DNS UNKNOWN record object.
- `recordnamepolicy` : Record name policy object.
- `request` : WAPI handler object.
- `restartservicestatus` : Restart service status object.
- `rir` : Regional Internet Registry object.
- `rir:organization` : Regional Internet Registry organization object.
- `roaminghost` : DHCP Roaming Host object.
- `ruleset` : DNS Ruleset object.
- `saml:authservice` : SAML authentication service object.

- [scavengingtask](#) : DNS scavenging task object.
- [scheduledtask](#) : Scheduled Task object.
- [search](#) : Search object.
- [sharednetwork](#) : DHCP Shared Network object.
- [sharedrecord:a](#) : DNS Shared A record object.
- [sharedrecord:aaaa](#) : DNS Shared AAAA record object.
- [sharedrecord:cname](#) : DNS Shared CNAME record object.
- [sharedrecord:mx](#) : DNS Shared MX record object.
- [sharedrecord:srv](#) : DNS Shared SRV record object.
- [sharedrecord:txt](#) : DNS Shared TXT record object.
- [sharedrecordgroup](#) : DNS Shared Record Group object.
- [smartfolder:children](#) : Smart Folder children object.
- [smartfolder:global](#) : Global Smart Folder object.
- [smartfolder:personal](#) : Personal Smart Folder object.
- [snmpuser](#) : SNMP user object.
- [superhost](#) : SuperHost object.
- [superhostchild](#) : Super Host Child object.
- [syslog:endpoint](#) : The syslog endpoint object.
- [tacacsplus:authservice](#) : The TACACS+ authentication service object.
- [taxii](#) : Taxii Member object.
- [tftpfiledir](#) : TFTP file or directory object.
- [threatinsight:allowlist](#) : Threat insight allowlist object.
- [threatinsight:cloudclient](#) : Threat Insight Cloud Client object.
- [threatinsight:insight_allowlist](#) : Threat insight allowlist object.
- [threatinsight:moduleset](#) : Threat insight module set object.
- [threatprotection:grid:rule](#) : Threat protection custom rule object.
- [threatprotection:profile](#) : The Threat Protection profile object.
- [threatprotection:profile:rule](#) : Threat Protection Profile Rule object.
- [threatprotection:rule](#) : Member Threat Protection Rule object.
- [threatprotection:rulecategory](#) : Threat protection rule category object.
- [threatprotection:ruleset](#) : The Grid threat protection ruleset object.
- [threatprotection:ruletemplate](#) : Threat protection rule template object.
- [threatprotection:statistics](#) : Threat protection statistics object.
- [upgradegroup](#) : Upgrade group object.
- [upgradeschedule](#) : Upgrade schedule object.
- [upgradestatus](#) : The upgrade status object.
- [userprofile](#) : User profile object.
- [vdiscoverytask](#) : Discovery task object.
- [view](#) : DNS View object.
- [vlan](#) : VLAN object.
- [vlanrange](#) : VLAN Range object.
- [vlanview](#) : VLAN View object.
- [zone_auth](#) : DNS Authoritative Zone object.
- [zone_auth_discrepancy](#) : Zone discrepancy information object.
- [zone_delegated](#) : DNS Delegated Zone object.
- [zone_forward](#) : DNS Forward Zone object.
- [zone_rp](#) : DNS Response Policy Zone object.
- [zone_stub](#) : DNS Stub Zone object.

Structs

- [ad_auth_server](#) : Active Directory Authentication Server.
- [addressac](#) : Address ac.
- [admingroup:adminsetcommands](#) : Set commands
- [admingroup:adminshowcommands](#) : Show commands.
- [admingroup:admintoplevelcommands](#) : Toplevel commands.
- [admingroup:cloudsetcommands](#) : Set commands.
- [admingroup:cloudshowcommands](#) : Show commands.
- [admingroup:databasesetcommands](#) : Set commands
- [admingroup:databaseshowcommands](#) : Show commands.
- [admingroup:dhcpsetcommands](#) : Set commands.
- [admingroup:dhcpshowcommands](#) : show commands.
- [admingroup:dnssetcommands](#) : Set commands.
- [admingroup:dnsshowcommands](#) : Show commands.
- [admingroup:dnstoplevelcommands](#) : Toplevel commands.
- [admingroup:dockersetcommands](#) : Set commands.
- [admingroup:dockershowcommands](#) : Show commands.
- [admingroup:gridsetcommands](#) : Set commands.
- [admingroup:gridshowcommands](#) : Show commands.
- [admingroup:licensingsetcommands](#) : Set commands.
- [admingroup:licensingshowcommands](#) : Show commands.
- [admingroup:lockoutsetting](#) : lockout_setting
- [admingroup:machinecontroltoplevelcommands](#) : Toplevel commands.
- [admingroup:networkingsetcommands](#) : Set commands.
- [admingroup:networkingshowcommands](#) : Show commands.
- [admingroup:passwordsetting](#) : Password settings.
- [admingroup:samlsetting](#) : SAML settings.
- [admingroup:securitysetcommands](#) : Set commands.
- [admingroup:securityshowcommands](#) : Show commands.
- [admingroup:troubleshootingtoplevelcommands](#) : Toplevel commands.
- [adsites](#) : Microsoft Server AD Sites.
- [awsrte53recordinfo](#) : 'Aws Rte53 Record Info.
- [awsrte53task](#) : AWS Route53 task.
- [awsrte53zoneinfo](#) : 'Aws Rte53 Zone Info.
- [azurednstask](#) : Azure DNS task.
- [bgpas](#) : BGP (Border Gateway Protocol) Autonomous System (AS)
- [bgpneighbor](#) : BGP (Border Gateway Protocol) Neighbor.
- [capacityreport:objectcount](#) : Type count struct.
- [captiveportal:file](#) : Captive portal file.
- [changedobject](#) : Changed object information.
- [ciscoise:eaassociation](#) : Cisco ISE extensible attribute association struct.
- [ciscoise:publishsetting](#) : Cisco ISE publish settings struct.
- [ciscoise:subscribesetting](#) : Cisco ISE subscribe settings struct.
- [clientsubnetdomain](#) : The client subnet domain structure.
- [dhcpddns](#) : Ddns Zone Primary.
- [dhcpmember](#) : Grid member serving DHCP.
- [dhcption](#) : DHCP option.
- [dhcpserver](#) : Microsoft Server DHCP Properties struct for MsServer Object

- [discovery:advancedpollsetting](#) : The advanced polling settings structure.
- [discovery:advancedsdnpollsettings](#) : The advanced SDN polling settings structure.
- [discovery:advisorsetting](#) : Advisor Settings structure.
- [discovery:autoconversionsetting](#) : This struct contains settings for automatic conversion
- [discovery:basicpollsettings](#) : Basic Poll Settings.
- [discovery:basicsdnpollsettings](#) : Basic SDN Poll Settings.
- [discovery:clicredential](#) : CLI credential.
- [discovery:conversion_attributes](#) : Discovery Data Conversion attributes.
- [discovery:device:portstatistics](#) : Port statistics.
- [discovery:devicedatacollectionstatus](#) : Device Data Collection Status struct.
- [discovery:devicehint](#) : This struct contains Device Hint.
- [discovery:devicesupportinfo](#) : Device Support Info struct.
- [discovery:discoverydataconversionresult](#) : Discovery Data Conversion result.
- [discovery:ifaddrinfo](#) : IfAddr information.
- [discovery:jobprocessdetails](#) : Discovery Job Process Details.
- [discovery:networkdeprovisioninfo](#) : Network Deprovision Info.
- [discovery:networkinfo](#) : Network info.
- [discovery:port](#) : The discovery port structure.
- [discovery:port:config:adminstatus](#) : Port Config Admin Status.
- [discovery:port:config:description](#) : Port Config Description.
- [discovery:port:config:vlaninfo](#) : Port Config VLAN info.
- [discovery:port:control:info](#) : Port Control info.
- [discovery:port:control:taskdetails](#) : Port Control Task Details.
- [discovery:scaninterface](#) : The discovery scan interface structure.
- [discovery:sdnconfig](#) : The SDN/SDWAN configuration structure.
- [discovery:seedrouter](#) : The seed router structure.
- [discovery:snmp3credential](#) : SNMP v3 Credential.
- [discovery:snmpcredential](#) : SNMP Credential.
- [discovery:statusinfo](#) : Status information.
- [discovery:vlaninfo](#) : VLAN information.
- [discovery:vrfmappingrule](#) : This struct contains VRF Mapping Rule.
- [discoverydata](#) : Discovered data.
- [discoverytaskport](#) : The network discovery TCP port.
- [discoverytaskvserver](#) : VMWare discovery server.
- [dnsseckey](#) : DNSSEC Key.
- [dnsseckeyalgorithm](#) : DNSSEC Key Algorithm.
- [dnsseckeyparams](#) : DNSSEC Key parameters.
- [dnssectrustedkey](#) : DNSSEC Trusted Key.
- [dnsserver](#) : Microsoft server DNS.
- [dnstapsetting](#) : DNSTAP Setting.
- [dtc:health](#) : Health information.
- [dtc:monitor:snmp:oid](#) : DTC SNMP Monitor OID.
- [dtc:pool:consolidated_monitor_health](#) : Consolidated monitor health structure.
- [dtc:pool:link](#) : DTC Pool link.
- [dtc:query:result](#) : Query records.
- [dtc:server:link](#) : DTC Server link.
- [dtc:server:monitor](#) : DTC Server Monitor.
- [dtc:topology:rule:source](#) : DTC topology rule source.
- [dxl:endpoint:broker](#) : The Data Exchange Layer endpoint broker structure.
- [eaexpressionop](#) : Extensible attribute expression operand.

- `exclusionrange` : Exclusion range.
- `exclusionrangetemplate` : Exclusion range template.
- `expressionop` : Expression operand.
- `extensibleattributedef:descendants` : Descendants.
- `extensibleattributedef:listvalues` : List of values.
- `extserver` : External Server.
- `extsyslogbackupserver` : External syslog backup server.
- `federatedrealm` : Federated Realms
- `filetransfersetting` : File Transfer Setting.
- `filterrule` : Filter rule.
- `fireeye:alertmap` : FireEye Alert Map.
- `fireeye:rulemapping` : Fireeye Rule Mapping.
- `forwardingmemberserver` : Forwarding Member Server.
- `gcpdnstask` : Gcp DNS task.
- `gmcgroup:member` : gmcgroup member structure
- `grid:attackdetect` : DNS attack detection settings.
- `grid:attackmitigation` : DNS Attack Mitigation object.
- `grid:autoblackhole` : DNS Auto Blackhole settings.
- `grid:certverificationresult` : Openssl verification result for one certificate.
- `grid:cloudapi:gateway:config` : Gateway config.
- `grid:cloudapi:gateway:endpointmapping` : Endpoint mapping.
- `grid:cloudapi:info` : Cloud Information.
- `grid:cloudapi:user` : Cloud user.
- `grid:consentbannersetting` : Consent banner setting.
- `grid:cspapiconfig` : Csp Api config.
- `grid:cspgridsetting` : CSP Grid setting
- `grid:dns:fixedrrsetorderfqdn` : Fixed RRset order FQDN.
- `grid:informationalbannersetting` : Informational level banner setting.
- `grid:licensesubpool` : Licese sub-pool settings.
- `grid:lockoutsetting` : Lockout Security Setting
- `grid:loggingcategories` : Grid logging setting information.
- `grid:ntp` : The Grid Network Time Protocol (NTP) settings structure.
- `grid:responseratelimiting` : DNS Response Rate Limiting.
- `grid:restartbannersetting` : Restart Banner Setting.
- `grid:servicerestart` : Restart Setting.
- `grid:servicerestart:group:schedule` : Restart Group Schedule.
- `gridmember_soamname` : Per-master SOA MNAME Information.
- `gridmember_soaserial` : Per-master SOA Serial Information.
- `hotfix` : Upgrade process hotfix.
- `hsm:entrustnshield` : Entrust nShield Hardware Security Module.
- `hsm:safenet` : SafeNet Hardware Security Module.
- `hsm:thales` : Thales Hardware Security Module.
- `hsm:thalesluna` : Thales Luna Hardware Security Module.
- `interface` : IPv6/IPv4 interfaces settings.
- `ipv6networksetting` : IPv6 Network Settings.
- `ipv6setting` : IPv6 Settings.
- `lan2portsetting` : LAN2 Port Setting.
- `lanhaportsetting` : LAN/HA Port Settings.
- `ldap_eamapping` : The LDAP extensible attribute mapping.
- `ldap_server` : The LDAP server structure.

- `logicfilterrule` : Logic Filter rule.
- `lomnetworkconfig` : The LOM network configuration structure.
- `lomuser` : The Lights Out Management (LOM) user.
- `member:cspmembersetting` : CSP Member setting
- `member:dnsgluerecordaddr` : 'Member DNS glue record address.
- `member:dnsip` : Member DNS additional IP structure.
- `member:ntp` : The member Network Time Protocol (NTP) settings structure.
- `member:pnodeToken` : Physical Node Token.
- `memberserver` : Member Server.
- `memberservicecommunication` : Member Service Communication.
- `memberservicestatus` : Member Service Status.
- `mgmtportsetting` : MGMT Port Setting.
- `monitoreddomains` : Monitored domains.
- `msdhcption` : Microsoft DHCP Options.
- `msdhcpserver` : MS DHCP server.
- `msdnserver` : Msserver Server.
- `msserver:aduser` : Microsoft Server AD user.
- `msserver:aduser:data` : Active Directory User Data.
- `msserver:dcnsrecordcreation` : An Infoblox Active Directory Domain Controller object.
- `natsetting` : NAT Settings.
- `networkview:assocmember` : Network View Associated Members structure.
- `nodeinfo` : Node Info.
- `notification:rest:templateinstance` : Notification REST template instance.
- `notification:rest:templateparameter` : Notification REST template parameter.
- `notification:ruleexpressionop` : Notification rule expression operand.
- `ntpac` : The Network Time Protocol (NTP) access control item structure.
- `ntpaccess` : The Network Time Protocol (NTP) access control setting.
- `ntpkey` : The Network Time Protocol (NTP) authentication key structure.
- `ntpserver` : The Network Time Protocol (NTP) server structure.
- `nxdomainrule` : Rule of Ruleset.
- `objectschangetrackingsetting` : Objects changes tracking setting.
- `ocsp_responder` : OCSP Responder.
- `option60matchrule` : Option 60 Match Rule.
- `ospf` : OSPF Settings.
- `outbound:cloudclient:event` : Outbound cloudclient event type
- `parentalcontrol:abs` : Parental control additional blocking server(ABS).
- `parentalcontrol:mssp` : Parental control managed service provider (MSP) service.
- `parentalcontrol:nasgateway` : NAS gateway mobile security settings.
- `parentalcontrol:siteapimember` : Site member parental control settings.
- `parentalcontrol:sitemember` : Site member parental control settings.
- `parentalcontrol:spm` : Parental control policy management service (SPM).
- `physicalportsetting` : Physical Port Settings.
- `preprovision` : Pre-provisioning Settings.
- `preprovisionhardware` : Pre-provisioning Hardware Settings.
- `properties:blackoutsetting` : Blackout Setting.
- `queriesuser` : Queries user.
- `radius:server` : The RADIUS authentication server structure.
- `rdatasubfield` : Unknown Record Rdata subfield value.
- `remoteddnszone` : Remote DDNS Zone structure.
- `saml:idp` : The SAML Identity Provider structure.

- `scheduledbackup` : Scheduled backup settings.
- `servicestatus` : Node Service Status.
- `setting:atpoutbound` : Outbound settings for ATP events
- `setting:automatedtrafficcapture` : Automated Traffic Capture.
- `setting:dnsresolver` : DNS resolver Setting.
- `setting:dynamicratio` : Dynamic Ratio Setting for DTC Pool.
- `setting:email` : The email settings for the Grid member.
- `setting:httpproxyserver` : HTTP Proxy Server Setting.
- `setting:inactivelockout` : Inactivity Lockout settings.
- `setting:ipam:threshold` : IPAM Threshold Settings.
- `setting:ipam:trap` : IPAM Trap Settings.
- `setting:msserver` : Microsoft server settings structure.
- `setting:network` : Network settings.
- `setting:password` : Password settings.
- `setting:scavenging` : DNS scavenging settings.
- `setting:schedule` : Schedule Setting.
- `setting:security` : Security settings.
- `setting:securitybanner` : Security banner settings.
- `setting:snmp` : SNMP setting.
- `setting:syslogproxy` : Syslog proxy settings.
- `setting:trafficcapturechr` : Grid level Traffic capture chr setting.
- `setting:trafficcaptureqps` : Grid level traffic capture QPS setting.
- `setting:triggerrecdnslatency` : Grid level settings for traffic capture based on recursive
- `setting:triggerrecqueries` : Grid level settings for traffic capture based on recursive
- `setting:triggeruthdnslatency` : Member level settings for traffic based on authoritative DNS
- `setting:viewaddress` : Notify and query source settings.
- `smartfolder:groupby` : Smart Folder group by structure.
- `smartfolder:groupbyvalue` : Smart Folder group by value structure.
- `smartfolder:queryitem` : Smart Folder query item structure.
- `smartfolder:queryitemvalue` : Smart Folder query item value structure.
- `sortlist` : DNS Sortlist.
- `ssh_key` : 'List of ssh keys for a particular user.
- `subjectalternativenam` : Subject alternative name structure.
- `svcpams` : Svc Params structure.
- `syslog:endpoint:servers` : Syslog server structure
- `syslogserver` : Syslog server.
- `tacacsplus:server` : The TACACS+ server structure.
- `taxii:rpzconfig` : Taxii Member RPZ Configuration.
- `threatprotection:natport` : NAT Threat Protection Port.
- `threatprotection:natrule` : NAT Threat Protection Rule.
- `threatprotection:ruleconfig` : Threat protection rule configuration.
- `threatprotection:ruleparam` : Threat protection rule parameter.
- `threatprotection:statinfo` : Threat protection statistical information.
- `thresholdtrap` : The Grid SNMP threshold trap structure.
- `trapnotification` : The Grid SNMP trap notification structure.
- `trapreceiver` : Trap receiver.
- `tsigac` : TSIG ac.
- `updatesdownloadmemberconfig` : Updates Download Member Configuration.
- `upgradegroup:member` : Upgrade group member structure.
- `upgradegroup:schedule` : Upgrade schedule group structure.

- [upgradestep](#) : Upgrade process step.
- [vlanlink](#) : VLAN link.
- [vtftpdirmember](#) : Virtual TFTP directory member.
- [zoneassociation](#) : Zone association.
- [zoneassociations](#) : Zone associations.
- [zonenameserver](#) : Zone Name Server.
- [zonerolloverinfo](#) : The zone rollover information structure.

Additional Information and Examples

- [Infoblox Developer community](#)
- [Examples accessing WAPI using Curl](#)
- [Examples using object body requests](#)
- [Valid values for extensible attributes](#)
- [Extensible attributes inheritance](#)
- [Extensible attributes search](#)
- [Extensible attributes update](#)
- [Glossary](#)
- [Object restrictions](#)